



Ferrum Bellum

Documentación Técnica

Autores:

Daniel Durán García – 21.493.398-5

Martín Castillo Torrejón – 21.484.992-5

Fecha: 20 de junio 2025

Asignatura: Estructura de Datos

Carrera: Ingeniería en Tecnología de Información

Institución: Universidad Católica del Norte



Contenido

Introducción	2
Tecnologías Utilizadas	3
Estructura del Proyecto	4
Modo de Juego	5
Inteligencia artificial	6
Requisitos del sistema, Compilación y Ejecución	7
Requisitos del sistema	7
Compilación	7
Ejecución	7
Descripción del código	8
Estructura de Archivos y Módulos	8
Funciones de tablero y Movimiento	9
Interfaz Gráfica y Menús	10
IA y Toma de Decisiones	10
Diseño Técnico e Implementación de Algoritmos.....	12
Estructura de datos.....	12
Flujo Principal	12
IA avanzada (Dificultad “Difícil”)	13
Análisis de Tiempos de Decisión de la IA (Dificultad Difícil)	15
Justificación de la Función Heurística	16
Conclusión	17



Introducción

Este documento describe la implementación técnica del proyecto Ferrum Bellum, un juego por turnos en C++ con interfaz gráfica desarrollada en SFML. El juego está diseñado para ejecutarse en sistemas Windows usando el entorno de desarrollo MSYS2 con el perfil ucrt64.

Tecnologías Utilizadas

- Lenguaje: C++
- Librería gráfica: SFML 2.6.1 (versión dinámica)
- Entorno de compilación: MSYS2 (UCRT64)
- Compilador: g++
- IDE recomendado: Visual Studio Code

Estructura del Proyecto

A continuación, se detalla la estructura del proyecto Ferrum Bellum, indicando la función principal de cada carpeta o archivo.

Proyecto:

- .vscode → Archivos de configuración de Visual Studio Code.
- Dominio → Contiene archivos .h y .cpp de las Clases Tanques y Nodos.
- Fuentes → Fuente utilizada para los textos en la interfaz gráfica.
- Imágenes → Recursos gráficos usados (Sprites).
- Lógica → Contiene el archivo main.cpp, con la lógica del juego.
- Sonidos → Contiene los archivos de música usados en el juego.
- Documentacion_Ferrum_Bellum.pdf → Contiene la documentación del proyecto.
- README.md → Contiene información de los integrantes (nombre, rut, carrera).
- app.exe → Ejecutable generado tras compilar el proyecto.
- (Otros archivos) → Archivos diversos.

Modo de Juego

El flujo general del juego se desarrolla en las siguientes etapas:

1. Pantalla de inicio

El jugador comienza en una pantalla de inicio, donde deberá presionar cualquier tecla.

2. Menú Principal

Se despliega un menú con las siguientes opciones:

- Jugar: Inicia el proceso del juego
- Configuraciones: permite acceder a:
 - Ajuste del volumen.
 - Cambio de resolución de pantalla.
 - Restaurar valores por defecto.
 - Volver al menú anterior.
- Salir: Cierre el juego de forma inmediata.

3. Selección de dificultad

El jugador elige el nivel de dificultad de la inteligencia Artificial: Fácil, Media o Difícil.

4. Selección de Tanques y Posicionamiento Inicial.

El jugador selecciona tres tanques de diferentes tipos (Ligero, Mediano, Pesado) y define sus posiciones iniciales en el tablero.

5. Fase de combate

Se inicia el bucle de combates por turnos. En cada ronda, tanto el jugador como la IA eligen sus acciones (moverse o disparar), las cuales se ejecutan de formas simultánea.

6. Resultado de la Partida

Una vez finalizado el combate, se muestra una pantalla de victoria o derrota. Al presionar la tecla Esc, se regresa al menú principal, permitiendo reiniciar una nueva partida.

Inteligencia artificial

Nivel	Descripción
Nivel 1 (Básico)	Disparo basado solo en la posición del jugador. No considera vida ni movimientos. (Probabilidad simple).
Nivel 2 (Intermedio)	Considera posición y distancia. Decide si moverse o disparar. (Probabilidad + lógica simple).
Nivel 3 (Avanzado)	Considera todos los factores del juego. Implementa el algoritmo Minimax con poda alfa-beta.

Requisitos del sistema, Compilación y Ejecución

Requisitos del sistema

- Tener instalado MSYS2
- Instalar la librería SFML:
`pacman -S mingw-w64-ucrt-x86_64-sfml`

Compilación

Para compilar el proyecto, asegúrate de estar en la raíz del repositorio y ejecuta el siguiente comando en la terminal de MSYS2:

```
C:/msys64/ucrt64/bin/g++.exe -g Logica/main.cpp
Dominio/Tanques/Tanque.cpp Dominio/Tanques/TanquePesado.cpp
Dominio/Tanques/TanqueLigero.cpp Dominio/Tanques/TanqueMediano.cpp
Dominio/Nodos/NodoSistema.cpp -I C:/msys64/ucrt64/include -o
app.exe -L C:/msys64/ucrt64/lib -lsfml-graphics -lsfml-window -
lsfml-system -lsfml-audio -lopengl32 -lfreetype -lwinmm -lgdi32
```

Ejecución

Una vez que la compilación se haya realizado exitosamente y el archivo app.exe esté disponible, puedes ejecutar el juego con el siguiente comando:

```
./app.exe
```

Descripción del código

A continuación, se describe la funcionalidad de los principales componentes del código fuente del proyecto.

Estructura de Archivos y Módulos

- **main.cpp**

Archivo principal del proyecto. Contiene la función `main()`, que coordina el flujo completo del juego. Sus principales responsabilidades son:

- Establecer resolución y crear la ventana del juego.
- Cargar recursos (fuentes, texturas, sonidos).
- Mostrar menús: pantalla de inicio, menú principal, selección de dificultad.
- Gestionar la selección y posicionamiento de tanque del jugador.
- Asignar y posicionar tanques para la IA.
- Ejecutar el bucle de combate(`bucleDeCombate()`).
- Aplicar configuraciones (resolución y volumen).
- Liberar memoria y cerrar la aplicación al finalizar.

- **Carpeta Dominio**

Contiene las clases encargadas de modelar los tanques como el tablero de juego.

Clases de Tanques

- Tanque: clase abstracta base, define los atributos comunes (vida, daño, movimientoBase, id) y métodos como `actualizarVida()`, `getProbabilidadDelImpacto()`, `clonar()`.
- TanqueLigero: alta movilidad (6), daño bajo (100), vida 200.
- TanqueMediano: balanceado (movimiento 4, daño 150, vida 250).
- TanquePesado: movimiento lento (2), alto daño (200), vida 350.

Cada tipo de tanque implementa su propia versión de `clonar()` y `obtenerVidaPorTipo()`.

Clase NodoSistema

- Modela una celda del tablero como un nodo de una lista doblemente enlazada.
- Atributos:
 - posición (x, y): coordenadas que indican la ubicación de la celda dentro del tablero.

- tipoTerreno: entero que representa el tipo de terreno, afectando movilidad y probabilidad de impacto.
Codificación:
 - 0: Agua (inaccesible)
 - 1: Planicie (sin penalización)
 - 2: Bosque (penalización media)
 - 3: Montaña (penalización alta)
- Tanque: puntero al tanque contenido en esa celda (si hay uno).
- Siguiente / anterior: punteros que permiten recorrer la lista enlazada de nodos.
- Métodos:
 - Getters y setters para cada atributo.
 - destruirTanque(): elimina el tanque contenido (si existe) y libera la memoria correspondiente.
- **Estructuras Auxiliares (declaradas en main.cpp)**
 - Resolucion: define el tamaño de pantalla.
 - Explosion: registra explosiones activas (posición, tipo, tiempo)
 - MemoriaTactica: historial de movimiento del jugador, útil para IA.
 - EstadoJuego: representa un estado clonado del juego para simulación (tablero y tanques).
 - AccionPlanificada: estructura que almacena si un tanque se moverá o disparará, junto con coordenadas objetivo.

Funciones de tablero y Movimiento

- **Creación y manipulación del tablero**
 - crearTablero(): construye el tablero de 5×5 nodos con tipo de terreno.
 - agregarPosicion(): agrega nodos a la lista enlazada.
 - clonarTablero(): crea una copia profunda del tablero para la IA.
 - actualizarReferenciasTanques(): reconecta tanques clonados al tablero clonado.
- **Consultas y Movimientos**
 - buscarNodo(tablero, x, y): localiza un nodo por coordenadas.
 - esCeldaLibre(nodo): indica si la celda está vacía.
 - obtenerPosicionTanque(tablero, tanque): ubica el tanque en el tablero.
 - obtenerPatronMovimiento(tanque, tablero): retorna posibles movimientos del tanque.
- **Acciones**

- moverse(), disparar(): realizan movimientos/disparos reales.
- moverseSimulado(), dispararSimulado(): realizan simulaciones sin afectar el juego real.
- ejecutarMovimiento(), ejecutarDisparo(): ejecutan acciones planificadas en simulaciones.

Interfaz Gráfica y Menús

- mostrarPantallaInicio(): pantalla inicial, espera una tecla para avanzar.
- mostrarMenuPrincipal(): opciones Jugar, Configuración y Salir.
- mostrarMenuDificultad(): elige la dificultad de la IA.
- mostrarMenuSelecciónTanquesJugador(): selección e ingreso de coordenadas para tanques del jugador.
- mostrarMenuConfiguración(): cambia el volumen y la resolución.
- aplicarConfiguraciónPantalla(window): actualiza la ventana con la resolución elegida.
- menuAccionesJugadorSimultáneo(): interfaz de combate por turnos simultáneos (mover/disparar).
- desplegarTablero(): renderiza el tablero, tanques y explosiones.
- mostrarMensajeFinal(): muestra mensaje de victoria o derrota, espera tecla Esc para volver.

IA y Toma de Decisiones

IA Fácil

- planificarAcciónIAFácil(): dispara si el jugador está en rango visible, o se mueve al azar.

IA Media

- planificarAcciónIAMedia(): evalúa distancia y vida para decidir entre atacar o moverse.

IA Difícil

- generarHijosAvanzados(): simula todas las acciones posibles (movimientos y disparos).
- evaluarAmenazasTerritoriales(): puntuación táctica rápida de control de terreno.
- evaluarEstadoAvanzado(): función heurística completa basada en 10 criterios (vida, terreno, distancia, movilidad, posición, etc).

- `minimaxAvanzado()`: algoritmo Minimax con poda Alfa-Beta para explorar decisiones.
- `encontrarMejorJugadaAvanzada()`: busca la mejor acción evaluando tiempos, memoria y heurísticas.
- `obtenerAccionIA()`: decide qué IA usar según dificultad elegida.
- `predecirProximoMovimiento()`: estima la siguiente posición probable del jugador con base en su historial (usando la estructura `MemoriaTactica`), ayudando a anticipar sus movimientos.

Ejecución del Turno

- `seleccionarTanquesIA()`: asigna aleatoriamente tres tanques al equipo de la IA y los posciona en el tablero (filas 3 y 4), evitando superposición.
- `ejecutarAccionesSimultaneas()`: ejecuta las acciones del jugador y la IA al mismo tiempo. Detecta conflictos y aplica daño.

Combate

- `bucleDeCombate()`: contiene el ciclo principal de juego. Termina cuando un bando pierde todos sus tanques o el jugador cierra la ventana.

Diseño Técnico e Implementación de Algoritmos

Esta sección resume el diseño general del motor del juego, su arquitectura de datos y el enfoque algorítmico utilizado, con énfasis en la implementación de la IA Difícil.

Estructura de datos

- **Lista doblemente enlazada (NodoSistema)**
Representa el tablero de 5×5. Cada nodo contiene su posición, tipo de terreno y un puntero al tanque que puede estar presente. Permite recorrer el tablero de forma eficiente en ambas direcciones.
- **Pilas de tanques (std::stack<Tanque*>)**
Se utiliza para almacenar los tanques del jugador y de la IA. Esta estructura garantiza el orden de inserción y facilita el recorrido por turnos.
- **Clonación de estado (EstadoJuego)**
En la IA difícil, se clona el estado actual del juego para simular diferentes decisiones sin afectar el juego real. Esto incluye:
 - Copia del tablero (clonarTablero()).
 - Referencias internas reestablecidas (actualizarReferenciasTanques()).
 - acción planificada (AccionPlanificada).
- **Memoria táctica (MemoriaTactica)**
Registra patrones anteriores del jugador, permite que la IA tome decisiones más coherentes con el historial de juego.

Flujo Principal

1. **Inicio (main())**
 - Configura la ventana según la resolución seleccionada.
 - Carga recursos (texturas, fuentes, música)
 - Muestra menús: inicio → principal → dificultad → selección tanques → combate → victoria/derrota.
 - Al terminar, libera memoria.
2. **Funciones de Interfaz**
 - Permite navegación entre menús, configuración del entorno y acciones de combate de forma visual e intuitiva.
3. **Generación del Tablero**

- `crearTablero()` y `agregarPosicion()`: crean la estructura enlazada del mapa de juego
- Cada celda es única, y puede contener diferentes tipos de terreno, lo cual impacta la movilidad y precisión de disparo.

4. Sistema de Acciones

- Tanto los movimientos como los disparos se planifican con estructuras específicas y se ejecutan de forma sincrónica.
- Las versiones simuladas permiten a la IA explorar distintas posibilidades sin afectar el juego real.

5. Asignación de Tanques de la IA

- `seleccionarTanquesIA()` distribuye automáticamente los tres tanques del equipo enemigo (IA) en las filas 3 y 4 del tablero, evitando solapamientos y asegurando que se posicionen estratégicamente desde el inicio.

IA avanzada (Dificultad “Difícil”)

La IA difícil implementa un enfoque de búsqueda adversarial utilizando el algoritmo Minimax con poda alfa-beta, lo que le permite:

- Simular escenarios de varios turnos hacia adelante.
- Evaluar múltiples opciones antes de actuar.
- Adaptarse al contexto del juego (ventaja, desventaja, empate).

Componentes Clave:

- **`minimaxAvanzado()`**
Recorre el árbol de acciones posibles hasta una profundidad determinada, evaluando cada estado según su calidad. La poda alfa-beta reduce la cantidad de ramas analizadas al eliminar aquellas que no aportan mejor resultado.
- **`evaluarEstadoAvanzado()`**
Heurística que calcula el valor de un estado combinando múltiples factores:
 - diferencia de vida entre bandos.
 - Tipo de terreno ocupado.
 - Distancia entre tanques.
 - Control del centro o zonas clave del tablero.
 - Movilidad restante.
 - Patrón de movimiento enemigo y experiencia previa.
- **`evaluarAmenzasTerritoriales()`**

Sistema rápido para evaluar control territorial en simulaciones, útil para ordenar las mejores jugadas primero y así favorecer la poda alfa-beta.

- **predecirProximoMovimiento()**

Estima la próxima posición del jugador usando patrones guardados en MemoriaTactica, ayudando a anticipar movimientos y mejorar la planificación de disparos y desplazamientos.

- **Profundidad Adaptativa**

La IA modifica dinámicamente la profundidad del análisis según contexto:

- Aumenta profundidad si está en desventaja.
- Reduce profundidad para jugar más rápido si está ganando.

Análisis de Tiempos de Decisión de la IA (Dificultad Difícil)

Se midió el tiempo que tarda la IA en encontrarMejorJugadaAvanzada():

Contexto	Tiempo (ms)
Primera jugada	56
Máximo en igualdad	426
Media en igualdad	380
Media en ventaja (IA)	100
Media en desventaja	150

- Primera jugada: menor carga de patrones y menor profundidad adaptativa.
- Igualdad de vida: profundidad base y generación completa de estados, por eso valores medios altos (~380ms).
- Ventaja de vida: IA ajusta profundidad a +1, genera menos hijos y poda más agresiva, reduciendo el tiempo (~100ms).
- Desventaja: IA profundizada +2 y evalúa más caminos defensivos, por eso sube a ~150ms.

Justificación de la Función Heurística

La función evaluarEstadoAvanzado(estado) asigna puntos en base a factores ponderados:

1. Diferencia de vida ($\text{vida} \times 1000$)
 - Un gran multiplicador porque es condición principal de victoria/derrota
2. Bonificación por vida propia ($\text{vidaIA} \times 200$)
 - Refuerza mantener tanques con alta vida.
3. Penalización por vida enemiga ($-\text{vidaJugador} \times 180$)
 - Fomenta reducir la vida del rival.
4. Ventaja de terreno ($(\text{terrenoIA} - \text{terrenoJugador}) \times 50$)
 - Cada tipo de terreno aporta un valor extra, midiendo cuan favorable es la casilla.
5. Evaluación territorial avanzada ($\text{evaluarAmenazasTerritoriales} \times 1$)
 - Control de celdas adyacentes: suma 3 puntos por celda de ventaja y 8 puntos por rutas de escape, equilibrando agresión y seguridad.
6. Control posicional dinámico ($+(\text{centroJugador} - \text{centroIA}) \times 25$)
 - Prioriza ocupar el centro cuando IA domina, o esquinas si pierde.
7. Distancia estratégica
 - Dominando ($-|\text{dist}-2| \times 20$): busca distancia optima de ataque a 2 celdas.
 - Ligera ventaja ($-\text{dist} \times 15$): acércate
 - Empate ($+30$ si $\text{dis}=2$): controla distancia media.
 - Desventaja ($+\text{dist} \times 10$ ó 25): incrementar distancia a rivales peligrosos.
8. Exposición a ataques (± 150 ó -300)
 - Si IA está a distancia 1 y en desventaja, penaliza fuertemente para evitar suicidios; si domina, premia el ataque cercano.
9. Movilidad ($(\text{movilidadIA} - \text{movilidadJugador}) \times 40$)
 - Aprecia cuantas opciones de desplazamiento quedan.
10. Experiencia acumulada ($+5 \times \text{exitos}$ y $-2 \times \text{fallos}$)
 - Ajusta la IA para “aprender” de disparos exitosos o fallidos.

Conclusión

El proyecto Ferrum Bellum integra de forma coherente estructuras de datos dinámicas, un motor de IA basado en Minimax con poda Alfa-Beta y una interfaz gráfica fluida desarrollada con SFML. Gracias a la modularidad del código (clases de dominio, manejo de memoria, generación de tablero y capas de simulación), se logra un diseño sólido y fácil de mantener. La IA de nivel "Difícil" demuestra un balanceo efectivo entre complejidad táctica y rendimiento, tomando decisiones en promedio por debajo de 400 ms y adaptando su profundidad de búsqueda en función de la ventaja o desventaja en el combate. La función heurística, fundamentada en diez criterios ponderados (vida, terreno, combate territorial, movilidad, entre otros), permite a la IA priorizar correctamente el ataque, la defensa y el posicionamiento en el tablero. En conjunto, Ferrum Bellum fue un gran proyecto para la aplicación de conceptos de estructuras de datos y algoritmos, dejando una base sólida para futuras implementaciones.